

Task 0: Lab Preparation

```
Oct 4 16:30
ubuntu@ubuntu2204: ~/csce465-agentsec

GROUP: ffdhe3072
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec$ python3 --version
Python 3.12.3
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec$ openssl version
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec$ pip show cryptography pytest
Name: cryptography
Version: 49.0.0
Summary: cryptography is a package which provides cryptographic recipes and primitives to Python developers.
Home-page: https://github.com/pyca/cryptography
Author: The Python Cryptographic Authority and individual contributors <cryptography-dev@python.org>
License-Expression: Apache-2.0 OR BSD-3-Clause
Location: /home/ubuntu/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: cffi
Required-by:
...
Name: pytest
Version: 9.1.1
Summary: pytest: simple powerful testing with Python
Home-page: https://docs.pytest.org/en/latest/
Author: Brianna Laughner, Bruno Oliveira, Floris Bruynooghe, Freya Bruhin, Holger Krekel, Others (See AUTHORS), Ronny Pfannschmidt
Author-email:
License-Expression: MIT
Location: /home/ubuntu/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: iniconfig, packaging, pluggy, pygments
Required-by:
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec$
```

Task 1:

```
Oct 4 19:44
ubuntu@ubuntu2204: ~/csce465-agentsec/hw2

(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ python baseline_ctr.py
Original plaintext:
b'{"action":"READ","path":"notes.txt"}'

Original ciphertext:
95b91c65773b1036276d89f31db66ec13df6275188e05794a5ceb413d28c473041283e42

Original decrypted ciphertext:
b'{"action":"READ","path":"notes.txt"}'

XOR:
R XOR E = 0x17
E XOR C = 0x6
A XOR H = 0x9
D XOR O = 0xb

Tampered ciphertext:
95b91c65773b1036276d89e41bbf65c13df6275188e05794a5ceb413d28c473041283e42

Tampered ciphertext decrypted:
b'{"action":"ECHO","path":"notes.txt"}'

Replay:
Receiver processed:
b'{"action":"READ","path":"notes.txt"}'
Receiver processed:
b'{"action":"READ","path":"notes.txt"}'
No replay protection as the same ciphertext is accepted and decrypted twice
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ S
```

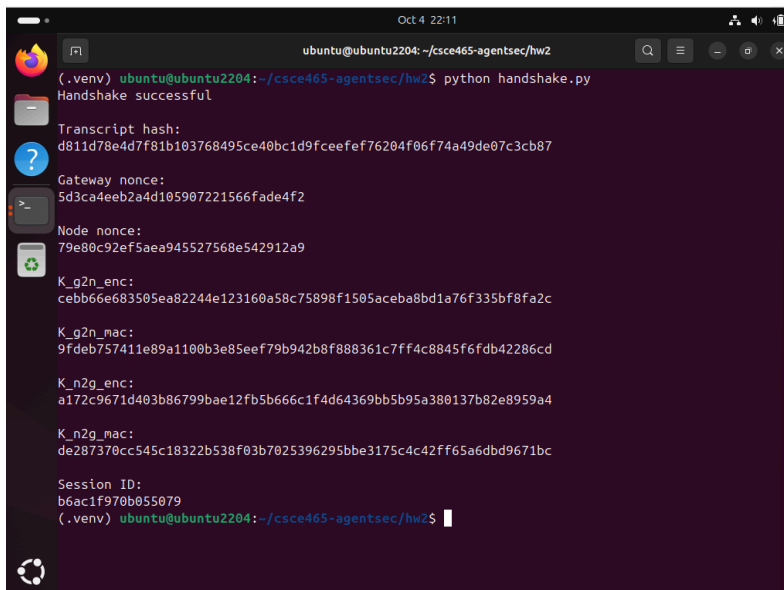
Design: I used AES-CTR to encrypt the plaintext `{"action":"READ","path":"notes.txt"}`. I then made a relay that changed the ciphertext so that a READ command was changed to an ECHO command. I also made the receiver show that the same ciphertext was accepted and processed twice.

Results of Experiment: The results of the experiment showed how the original `{"action":"READ","path":"notes.txt"}` plaintext was encrypted, the ciphertext was tampered with, and the tampered ciphertext showed no proof of tampering. The receiver

is also shown to have no ability to check whether a ciphertext was processed previously processed or accepted.

Root Cause: The root cause is the nature of AES-CTR. CTR uses XOR for encryption, which means changes to the ciphertext cause predictable changes to the decrypted plaintext. There is no MAC, so changes to the ciphertext in transit from sender to receiver cannot be spotted and a message cannot be verified as untampered with. The receiver also doesn't keep track of the ciphertexts it's seen before.

Task 2:



```
Oct 4 22:11
ubuntu@ubuntu2204: ~/csce465-agentsec/hw2
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ python handshake.py
Handshake successful

Transcript hash:
d811d78e4d7f81b103768495ce40bc1d9fceeef76204f06f74a49de07c3cb87

Gateway nonce:
5d3ca4eeb2a4d105907221566fade4f2

Node nonce:
79e80c92ef5aea945527568e542912a9

K_g2n_enc:
cebb66e683505ea82244e123160a58c75898f1505aceba8bd1a76f335bf8fa2c

K_g2n_mac:
9fdeb757411e89a1100b3e85eef79b942b8f888361c7ff4c8845f6fdb42286cd

K_n2g_enc:
a172c9671d403b86799bae12fb5b666c1f4d64369bb5b95a380137b82e8959a4

K_n2g_mac:
de287370cc545c18322b538f03b7025396295bbe3175c4c42ff65a6dbd9671bc

Session ID:
b6ac1f970b055079
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$
```

I used Diffie Hellman to create a handshake between a node and a gateway. Each side has a 3072 bit RSA key for signing. Each session, both sides generate a new Diffie Hellman private value and use a nonce (number used once) that's 16 bytes. Both sides then build a transcript that has the protocol name, group name, both identities, both public Diffie Hellman values, and both nonces. The transcript is resistant against misinterpretation as it follows a format. Both sides then hash and sign the transcript. Because of the signatures, the scheme provides authentication, meaning if someone were to edit the values in the transcript, it would be detected and the verification would fail. Additionally, the use of nonces means that each handshake is unique, preventing replay attacks. Without signatures, it allows someone to attack the scheme with a man in the middle attack as there would be no verification for the handshake contents.

Task 3:

```
Oct 4 22:29
ubuntu@ubuntu2204: ~/csce465-agentsec/hw2
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ python secure_record.py
Gateway to node:
b'hello node'

Message type:
1

Gateway sequence:
1

Node expected sequence:
1

Node to gateway:
b'hello gateway'

Message type:
1

Node sequence:
1

Gateway expected sequence:
1
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$
```

Task 4:

```
Oct 4 22:41
ubuntu@ubuntu2204: ~/csce465-agentsec/hw2
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ python -m pytest -v
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0 -- /home/ubuntu/csce465-agentsec/.venv/bin/python
cachedir: .pytest_cache
rootdir: /home/ubuntu/csce465-agentsec/hw2
collected 11 items

tests/test_security.py::test_valid_handshake_and_messages PASSED [ 9%]
tests/test_security.py::test_modified_ciphertext_rejected PASSED [ 18%]
tests/test_security.py::test_modified_header_rejected PASSED [ 27%]
tests/test_security.py::test_replay_rejected PASSED [ 36%]
tests/test_security.py::test_reflected_record_rejected PASSED [ 45%]
tests/test_security.py::test_wrong_rsa_key_rejected PASSED [ 54%]
tests/test_security.py::test_changed_nonce_rejected PASSED [ 63%]
tests/test_security.py::test_changed_public_value_rejected PASSED [ 72%]
tests/test_security.py::test_reflected_handshake_rejected PASSED [ 81%]
tests/test_security.py::test_malformed_transcript_rejected PASSED [ 90%]
tests/test_security.py::test_unexpected_identity_rejected PASSED [100%]

===== 11 passed in 2.54s =====
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$
```

Security Report: The attacker model assumes the attacker is between the gateway and the node. The attacker has the opportunity to intercept and observe/modify messages, inject new messages, replay messages, and reflect messages. The purpose of the scheme is to protect the confidentiality and integrity of the messages being sent and to authenticate the people involved. It does not protect individual devices from being hacked or attacked by compromised endpoints. It also doesn't check if a message is safe or not. This scheme can only verify the message sender and the integrity of its contents, meaning if it was modified or not. CTR requires every IV to be unique when the same encryption key is being used when the same encryption key is being used. This is because CTR makes a keystream based on the encryption key and the IV encrypted together. If the same key is used and the same IV is used, the XOR is the same, meaning the same keystream is produced. This is a vulnerability, as it can expose patterns and trends between plaintexts. The scheme prevents this by using a 16 byte shifting IV where the first eight bytes are the session ID and the last 8 bytes are increasing incrementally. This ensures that the same IV isn't used with the same key. For validation before decrypting, it's important, as we need to ensure the package/message wasn't tampered with before decrypting its contents. This prevents tampered and possibly malicious messages from being processed by the machine. By validating first, the system can safely ensure that the contents haven't been modified and process the contents. Independent encryption is used to ensure different processes or message directions are encoded with different keys. This means that keys aren't reused for all processes and that if one key is exposed or compromised, other parts of the process aren't affected. MAC keys are used to validate message contents. A sender signs the message with a HMAC tag using a MAC key and then the receiver validates the message using the same MAC key. If the results match, the message and its contents are validated. This encrypt then MAC system is different from the AEAD system. AES-GCM uses an encrypt and MAC in one construction, not two. This means it is able to accomplish the same goals as AES-CTR without an additional layer of HMAC. A valid protected record can still contain malicious commands because of the fact on how authentication works. For example, an authenticated node could legitimately send a command asking the gateway to delete an important file. The HMAC could verify successfully, the sequence number could be correct, and the message could come from the expected node. The cryptography would therefore correctly accept the record because it was not modified and came from an authenticated party. However, the command itself could still be dangerous. The endpoint must separately decide whether the authenticated sender is authorized to perform that specific action and whether the action is safe. Because of this concept, a verified and encrypted package can still contain a malicious tool call.